Poufna wymiana informacji przy pomocy oprogramowania typu *PGP*

Paweł Topa

28 września 2026

Wymagania

- Znajomość podstawowych komend systemu Linux/Unix (praca w terminalu, zmienne środowiskowe).
- Znajomość pojęć: szyfr symetryczny, szyfr asymetryczny, funkcja skrótu, podpis cyfrowy.

Literatura

- Historia oprogramowania PGP (*Pretty Good Privacy*): `https://pl.wikipedia.org/wiki/Pretty_Good_Privacy`
- Standard OpenPGP: `https://www.openpgp.org/` (RFC 4880, aktualnie RFC 9580)
- GnuPG – implementacja standardu OpenPGP: `https://www.gnupg.org/documentation/manuals/gnupg/` oraz FAQ: `https://www.gnupg.org/faq/gnupg-faq.html`
- Zdecentralizowana Infrastruktura Klucza Publicznego (*Web-of-Trust*): `https://en.wikipedia.org/wiki/Web_of_trust`
- Klucze sprzętowe YubiKey oraz GnuPG:
  - Yubico, *Using Your YubiKey with OpenPGP*: `https://support.yubico.com/s/article/Using-Your-YubiKey-wit`
  - Podręcznik GnuPG, opis `--list-secret-keys` (oznaczenia `sec>`, `ssb>`, `sec#`): `https://www.gnupg.org/documentation/manuals/gnupg/Operational-GPG-Commands.html`
  - *The GnuPG Smartcard HOWTO* (generowanie kluczy na karcie, `keytocard`): `https://www.gnupg.org/howtos/card-howto/en/smartcard-howto.html`
  - drduh, *YubiKey-Guide*: `https://github.com/drduh/YubiKey-Guide`
- Dokumentacja polecenia `gpg`: `man gpg`, `gpg --help`
- Budowa klucza OpenPGP – klucz główny i podklucze:
  - The GNU Privacy Handbook, rozdz. 1 *Getting Started*: `https://www.gnupg.org/gph/en/manual/c14.html`
  - Debian Wiki, *Subkeys*: `https://wiki.debian.org/Subkeys`
  - RFC 4880, sekcja 12.1 *Key Structures*: `https://www.rfc-editor.org/rfc/rfc4880#section-12.1`
  - RFC 9580, sekcja 5.2.3.29 *Key Flags*: `https://www.rfc-editor.org/rfc/rfc9580#section-5.2.3.29`
- Wyprowadzanie klucza z hasła – mechanizm S2K (*String-to-Key*):
  - RFC 9580, sekcja 3.7: `https://www.rfc-editor.org/rfc/rfc9580#section-3.7`
  - RFC 4880, sekcja 3.7: `https://www.rfc-editor.org/rfc/rfc4880#section-3.7`
  - opcje `--s2k-mode`, `--s2k-count`, `--s2k-digest-algo` w podręczniku GnuPG: `https://www.gnupg.org/documentation/manuals/gnupg/OpenPGP-Options.html`
  - funkcje wyprowadzania kluczy (KDF) – wprowadzenie: `https://en.wikipedia.org/wiki/Key_derivation_function`

Przygotowanie

- Ćwiczenie można wykonać na komputerach w laboratorium (system operacyjny macOS wraz z linią poleceń) lub na maszynach wirtualnych (oprogramowanie VirtualBox, maszyna Ubuntu, konto `ubuntuvm`, hasło `testvm`). Sprawdź wersję oprogramowania (`gpg --version`) i zapisz ją w pliku wyników.
- Przygotuj katalog (nazwij go swoim nazwiskiem) i umieść w nim plik `lab-results.txt`, w którym będziesz zapisywać użyte polecenia, ich wyniki oraz odpowiedzi na pytania z instrukcji.
- Aby nie mieszać swoich kluczy z kluczami innych użytkowników konta laboratoryjnego, utwórz osobną bazę kluczy i wskaż ją zmienną `GNUPGHOME` (zmienną ustaw w **każdym** nowo otwartym terminalu):
  `mkdir -m 700 ~/gnupg-lab ; export GNUPGHOME=~/gnupg-lab`
- Na koniec zajęć umieść w swoim katalogu wszystkie wytworzone pliki, skompresuj go i umieść jako wynik zadania w UPEL. **Nie umieszczaj w archiwum kluczy prywatnych ani haseł!**
- Po zakończeniu zajęć usuń katalog `~/gnupg-lab` z komputera w laboratorium.

1 Szyfrowanie danych kryptografią symetryczną

☐1 Zaszyfruj instrukcję tego ćwiczenia (dokument PDF) algorytmem symetrycznym AES z kluczem o długości 256 bitów (algorytm wskaż jawnie w poleceniu).

☐2 Sprawdź, jakim algorytmem faktycznie zaszyfrowano plik[1].

Pytanie: *klucz AES ma 256 bitów, a Ty podajesz hasło. Skąd bierze się klucz szyfrujący? (poszukaj hasła S2K)*

☐3 Odszyfruj plik do nowego pliku i sprawdź, czy jest identyczny z oryginałem (np. `cmp`).
Uwaga: program `gpg-agent` zapamiętuje hasło przez pewien czas, więc przy deszyfrowaniu możesz nie zostać o nie zapytany. Aby wymusić ponowne pytanie o hasło, użyj `gpgconf --kill gpg-agent`.

2 Obliczanie skrótu wiadomości

☐1 Wygeneruj dowolny plik tekstowy o długości co najmniej kilkuset bajtów.

☐2 Oblicz skrót SHA-256 pliku i zapisz go do pliku[2].

☐3 Dokonaj minimalnej edycji pliku tekstowego z punktu 1 (np. zmień jedną literę).

☐4 Ponownie oblicz skrót i dołącz go do pliku z punktu 2.

3 Kryptografia asymetryczna (z kluczem publicznym)

3.1 Tworzenie kluczy szyfrowania i podpisu cyfrowego

☐1 Przygotuj dwa zestawy kluczy: jeden do szyfrowania i podpisywania, a drugi tylko do podpisywania[3]. Użyj różnych komentarzy (lub adresów email), aby łatwo odróżnić oba klucze. Zanotuj dane identyfikacyjne kluczy:

	Szyfrowanie i podpis	**Tylko podpis**
Typ klucza (algorytm)	.....	.....
Imię i nazwisko	.....	.....
Adres email	.....	.....
Komentarz	.....	.....
Długość klucza / krzywa	.....	.....
Termin ważności	.....	.....
Odcisk palca (*fingerprint*)	....	....

Hasła chroniącego klucz prywatny nie zapisuj w instrukcji ani w pliku wyników.

[1] Polecenie `gpg --list-packets`.
[2] Polecenie `gpg --print-md` lub `sha256sum`/`shasum -a 256`.
[3] Użyj `gpg --full-generate-key` – polecenie `--gen-key` nie pozwala wybrać typu i długości klucza.

☐2 Wyświetl listę kluczy publicznych (`gpg -k`) i prywatnych (`gpg -K`). Zwróć uwagę na oznaczenia `[S]`, `[C]`, `[E]` oraz na podklucze (`sub`/`ssb`).

Pytanie: *dlaczego klucz „szyfrowanie i podpis" składa się z klucza głównego i podklucza? Dlaczego GnuPG nie pozwala utworzyć klucza głównego „tylko do szyfrowania"?*

☐3 Odszukaj certyfikat unieważnienia (*revocation certificate*) utworzony automatycznie dla każdego klucza (katalog `openpgp-revocs.d`). Do czego służy?

3.2 Wymiana kluczy publicznych

☐1 Wyeksportuj swój klucz publiczny do pliku w postaci tekstowej *ASCII armor*[4].

☐2 Wymień klucze publiczne z innym uczestnikiem (lub uczestnikami) zajęć i zaimportuj je do swojej bazy danych.

☐3 Sprawdź i zanotuj stan pól **trust** ..... i **validity** .....[5].

☐4 Zweryfikuj autentyczność otrzymanych kluczy: porównaj odcisk palca (`gpg --fingerprint`) z właścicielem klucza niezależnym kanałem (np. osobiście). Jak to zrobisz? .....[6]

Pytanie: *dlaczego samo przesłanie klucza (np. mailem) nie wystarcza do potwierdzenia jego autentyczności?*

☐5 Podpisz zweryfikowane klucze swoim kluczem „szyfrowanie i podpis"[7].

☐6 Ponownie sprawdź i zanotuj stan **trust** ..... i **validity** .....

Pytanie: *które pole się zmieniło, a które nie? Dlaczego?*

3.3 Szyfrowanie i podpisywanie plików

W tej części ćwiczenia studenci wymieniają się w parach zaszyfrowanymi i podpisanymi plikami. Odbiorcę i klucz podpisujący wskazuj odciskiem palca lub identyfikatorem klucza, a nie samym adresem email.

Nadawca:

☐1 Wykorzystując otrzymany klucz publiczny zaszyfruj dowolny plik.

☐2 Wykorzystując własny klucz prywatny podpisz dowolny plik na trzy sposoby: podpis dołączony (`--sign`), podpis w postaci czystego tekstu (`--clearsign`) i podpis odłączony (`--detach-sign`).

Pytanie: *czy plik podpisany opcją `--sign` jest zaszyfrowany? Sprawdź.*

☐3 Zaszyfruj i jednocześnie podpisz jeden plik.

☐4 Wyślij pliki do właściciela klucza publicznego.

Odbiorca:

☐1 Odszyfruj otrzymany zaszyfrowany plik.

☐2 Zweryfikuj poprawność podpisów dla otrzymanych plików (dla podpisu odłączonego potrzebny jest zarówno plik podpisu, jak i plik podpisany).

☐3 Zmień jeden znak w podpisanym pliku i ponownie zweryfikuj podpis.

Pytanie: *jak zmienił się komunikat programu? Jaki związek ma to z częścią 2?*

[4] Opcja `--armor` (`-a`). Format ten przypomina PEM (*Privacy Enhanced Mail*) – porównaj oba formaty i zapisz, czym się różnią.

[5] Interfejs tekstowy: `gpg --edit-key <id>` (pola `trust:` i `validity:`), wyjście komendą `quit`. **Trust** (*ownertrust*) to Twoja ocena, na ile właściciel klucza rzetelnie weryfikuje cudze klucze; **validity** to wyliczona przez GnuPG pewność, że klucz należy do osoby wskazanej w identyfikatorze.

[6] Poszukaj informacji o zjawisku zwanym *key-signing party*.

[7] Masz dwa klucze prywatne – wskaż klucz podpisujący opcją `-u <id>`. Interfejs tekstowy: aby zmiany zostały zapisane w bazie danych należy użyć komendy `save`.

4 Sprzętowe klucze kryptograficzne YubiKey i GnuPG

☐1 Zapoznaj się z technologią kluczy kryptograficznych YubiKey.

☐2 Uzyskaj od prowadzącego klucz kryptograficzny YubiKey (PIN-y domyślne: user: `123456`, admin: `12345678`). *Uwaga:* trzykrotne podanie błędnego PIN-u blokuje go, a trzykrotne podanie błędnego PIN-u administratora blokuje aplikację OpenPGP na urządzeniu.

☐3 Zapoznaj się z komendami dotyczącymi operacji na kluczu YubiKey (`gpg --card-status`, `gpg --card-edit`, komenda `admin`).

☐4 Jeśli na urządzeniu znajdują się klucze poprzedniego użytkownika, przywróć ustawienia fabryczne (komenda `factory-reset` w trybie `admin`).

☐5 Utwórz na kluczu YubiKey zestaw kluczy publiczny/prywatny (komenda `generate`).

☐6 Wyświetl listę kluczy prywatnych (`gpg -K`).

Pytanie: *co oznacza znak* `>` *przy* `sec>/ssb>`*? Gdzie fizycznie znajduje się klucz prywatny?*

☐7 Wyeksportuj nowy klucz publiczny, przekaż go partnerowi i powtórz czynności z części 3.2 i 3.3, posługując się kluczami zapisanymi na urządzeniu.

☐8 Sprawdź, że odszyfrowanie i podpisanie wiadomości nie jest możliwe, gdy klucz jest odłączony od komputera.

☐9 Przed oddaniem urządzenia przywróć jego ustawienia fabryczne.

5 Zadanie dodatkowe: Infrastruktura Klucza Publicznego (PKI): model *Web-of-Trust*

Zakładamy, że warunki pracy nie pozwalają na bezpośrednią weryfikację kluczy. Konieczne jest pośrednictwo osoby trzeciej (nazwijmy ją Wielkim Bratem), której ufamy, że dokładnie sprawdziła autentyczność kluczy innych użytkowników. Autentyczność kluczy potwierdzona jest podpisem tej osoby.

Rolę Wielkiego Brata będzie pełnił prowadzący.

☐1 Poszukaj informacji o pojęciu ***Web-of-Trust*** w kontekście oprogramowania PGP/OpenPGP/GnuPG.

☐2 Pobierz z UPEL klucz Wielkiego Brata i zaimportuj go do swojej bazy danych. Porównaj jego odcisk palca z odciskiem podanym przez prowadzącego i podpisz go lokalnie (`--lsign-key`).

☐3 Nadaj kluczowi Wielkiego Brata pełne zaufanie (*trust*: *full*, w menu `trust` opcja „I trust fully")[8].

☐4 Udostępnij swój klucz Wielkiemu Bratu – zostanie podpisany i opublikowany (na UPEL).

☐5 Pobierz z UPEL klucz innego studenta, podpisany przez Wielkiego Brata (którego dotychczas nie masz w swojej bazie). Sprawdź i zanotuj **trust** ..... i **validity** ..... NIE PODPISUJ KLUCZA!

Pytanie: *dlaczego klucz jest ważny (validity), chociaż go nie podpisałeś(-aś)?*

☐6 Zmień zaufanie do klucza Wielkiego Brata na *marginal* i ponownie sprawdź **validity** klucza innego studenta.

Pytanie: *jak zmieniła się ważność klucza? Ile podpisów od osób z zaufaniem marginal jest domyślnie potrzebnych (opcja* `--marginals-needed`*)?*

[8] Poziom *ultimate* jest zarezerwowany dla Twoich własnych kluczy – klucz z tym poziomem GnuPG traktuje tak, jakby był Twój, nawet bez podpisu.